$ spring run FunctionalLab --level=0..100

Spring Boot × FP

// □ 0%□□□□□□□ 100%□□□□□□□□□

0%	25%	50%	75%	100%
□□□	Stream	□□□	Result	Core/Shell

$ cat refactor-plan.md

□□□□□□□□□□□□□□□□

0% / □□□
NPE□□□□□□□□□□□

25% / □□□□□
Stream □ Optional

50% / □□□
record □□□□□□□ mock

75% → 100% / □□□□
Result□□□□Core/Shell

□□□□□□□□□□□□□□□□□□□□ bug□

PART 01 — 0%

□□□□□□

□□□□□□□□□□□□

□□□□□□□

```java
@Service
public class OrderService {
    private double total; // □□□□□□

    public Order checkout(Long id) {
        Order order = repo.findById(id).get();
        total = 0;
        for (OrderItem item : order.getItems()) {
            if (item != null
                && item.getPrice() != null) {
                total += item.getPrice()
                       * item.getQty();
            }
        }
        if (order.getCoupon() != null) {
            // ...□□ if □□...
        }
        order.setTotal(total);
        emailService.send(order);
        return repo.save(order);
    }
}
```

SMELL REPORT

× `total` □□□□□ Service□□□□□□□
× □□ `.get()` □NPE □□□
× null □□□□□
× □□□□□□□□□□

□□□□□□□□□□

0% □□□□□

NPE □□□□
findById().get()□getPrice() □□□□ null

□□□□□□
□□□□ total□Service □□□□

□□□□□□
□□□□□ mock repository □ email

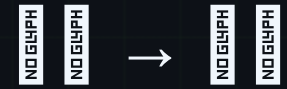
□ A □ B
□□□□□□I/O □□□□□

FP □□□□□□□□□□□□□ —— □□□□□□□

PART 02 — 25%

Stream NOGLYPH Optional

□□□□□□□ JDK □□□□



```java
double total = order.getItems().stream()
    .filter(Objects::nonNull)
    .mapToDouble(i -> i.getPrice() * i.getQty())
    .sum();
```

□□□
□□□□□□□□□□□□□□□□□

□□□
□□□□□□□□□□□□□□□□

null □□ → Optional □□

```java
Order order = orderRepository.findById(id)
    .orElseThrow(() -> new OrderNotFoundException(id));

double discounted = Optional.ofNullable(order.getCoupon())
    .filter(Coupon::isValid)
    .map(c -> total * 0.9)
    .orElse(total);
```

□□□
□□ .get()□□□ null □□□□□□□

□□□□
order □□□□□□□□ I/O □□□□

map / filter □□ FP □ Functor□□□□□□□□□□□□□□

PART 03 — 50%

□□□□□□□□□□

□□□□□□□□□□□□□□□

□□□□□□□□□□

```java
public record Pricing(BigDecimal subtotal, BigDecima

public final class PricingRules { // □□□
    static Pricing price(
        List<OrderItem> items,
        Optional<Coupon> coupon) {

        BigDecimal subtotal = /* stream □□ */;
        BigDecimal total = coupon
            .filter(Coupon::isValid)
            .map(c -> subtotal.multiply(RATE_90))
            .orElse(subtotal);
        return new Pricing(subtotal, total);
    }
}
```

Service □□

```java
public Order checkout(Long id) {
    Order order = findOrder(id);
    Pricing pricing = PricingRules.price(
        order.items(), order.coupon());
    return repo.save(
        order.withTotal(pricing.total()));
}
```

□□□ → □□□ → □□□
record □□ setter□□□□□□ DB□□□□□□□□□

□□□□□

0% □□□

```
@SpringBootTest
class OrderServiceTest {
    @MockBean OrderRepository repo;
    @MockBean EmailService email;
    // ...□□□□□stub □□□...
    // □□□□□□□□□□
}
```

50% □□□

```
@Test
void □□□□□□□□() {
    var items = List.of(
        new OrderItem("□□",
            new BigDecimal("1000"), 1));

    Pricing p = PricingRules.price(
        items, Optional.of(validCoupon()));

    assertThat(p.total())
        .isEqualByComparingTo("900");
}
```

□ mock · □□□ · □□□

□□□□□

$ spring run FunctionalLab

● □□

0% □□□□□

0% 25% 50% 75% 100%

✗ □□ null □□□NPE □□□□

✗ setter □□□□□□□□□□□

✗ □□□□ I/O□□□□ mock □□

✗ □□□□□□□□□□□□□□□

✗ □□□□□□□□□□□□□□□□

PART 04 — 75%

Result □□□□□□□

□□□□□□□□□□□

□□□□□□

```java
public sealed interface Result<T> {
    record Ok<T>(T value) implements Result<T> {}
    record Err<T>(String reason) implements Result<T> {}
}

// □□□ switch □□□□□□□□□□□□□□
return switch (result) {
    case Ok<Pricing>(Pricing p) -> ResponseEntity.ok(p);
    case Err<Pricing>(String reason) ->
        ResponseEntity.badRequest().body(reason);
};
```

□□□□□□□□ try-catch□□□□□ —— □□□□□□□□□□□□□□□□□

□□□□ = □□□□□□

```java
static UnaryOperator<Pricing> memberDiscount(Member m) {
    return p -> m.isVip()
        ? p.withTotal(p.total().multiply(RATE_95))
        : p;
}

Function<Pricing, Pricing> rules =
    memberDiscount(member)
        .andThen(couponDiscount(coupon))
        .andThen(freeShippingOver(new BigDecimal("1000")));
```

□□□
□□□□□□□□ andThen □□

□□□□□□
□□□□□□□ FP □

PART 05 — 100%

Functional Core, Imperative Shell

□□□□

```
@Service
public class CheckoutService { // Shell
    public CheckoutResponse checkout(Long id) {
        Order order = repo.findById(id)   // I/O
            .orElseThrow(...);

        CheckoutDecision decision = CheckoutCore
            .decide(order.snapshot(), clock.instant(

        return switch (decision) {
            case Approved a -> {
                repo.save(a.pricedOrder());
                events.publish(new OrderPriced(a));
                yield CheckoutResponse.ok(a);
            }
            case Rejected r ->
                CheckoutResponse.fail(r.reason());
        };
    }
}
```

□□□□□□□

```
public final class CheckoutCore {
    public static CheckoutDecision decide(
        OrderSnapshot order, Instant now) {

        if (order.items().isEmpty())
            return new Rejected("□□□");

        Pricing pricing = /* □□□ */;
        return new Approved(
            order.withPricing(pricing));
    }
}
```

□□□□□□

1. □□□□□□□□□□□□□□
2. □□□□□□□□□□□□□□□□

0% vs 100%

□□	0%	100%
□□□□	mock □□□□□□□□	□□□ mock□□□
Null □□	NPE □□□	Optional + record □□□□
□□□□	□□□□□□□	□□□□□□□□□□
□□□□	□□□□□□□□□	□□□□□□□□□□
□□□□	□□□□□□□□	Result + switch □□
□□□□	□□□ if □□	□□□□□□□□□ andThen

□□□□

Spring □□□□□□□□□□□

□□□Spring □□□DI□□□□□□□□□FP □□□□@Transactional □□ Shell □□Core □□□□□□□□

□□ new □□□□□□□□

JVM □□□□□□□□□TLAB + □□ GC□□□□□□□□□□□□□□□□□□

□□□□ 100%□

□□□□□□□□□□25% □□□ NPE □□□50% □□□□□□ mock□

□□□□

25%
Stream □□□□Optional □□□

50%
record [illegible] mock

75%
Result □□□□□□andThen □□□

100%
□□□□□□□□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□

Refactor complete.

□□□□□□□□□ /springboot/fp-in-spring-boot

```
$ spring run FunctionalLab --level=100
stream → record → result → core/shell
```